

Decomposition

- The only way to avoid the repetition-of-information problem in the in_dep schema is to decompose it into two schemas – instructor and department schemas.
- Not all decompositions are good. Suppose we decompose

employee(ID, name, street, city, salary)
into
employee1 (ID, name)
employee2 (name, street, city, salary)

The problem arises when we have two employees with the same name
we cannot reconstruct
the original employee relation -- and so, this is a lossy decomposition.

The Problem of Lossy Decomposition:

The figure illustrates a situation where decomposing a relation results in the loss of information, making it impossible to fully reconstruct the original data.

- **Original Relation:** Suppose we have a relation called `employee(ID, name, street, city, salary)`.
- **Decomposition:** The relation is decomposed into two relations:
 1. `employee1(ID, name)`
 2. `employee2(name, street, city, salary)`

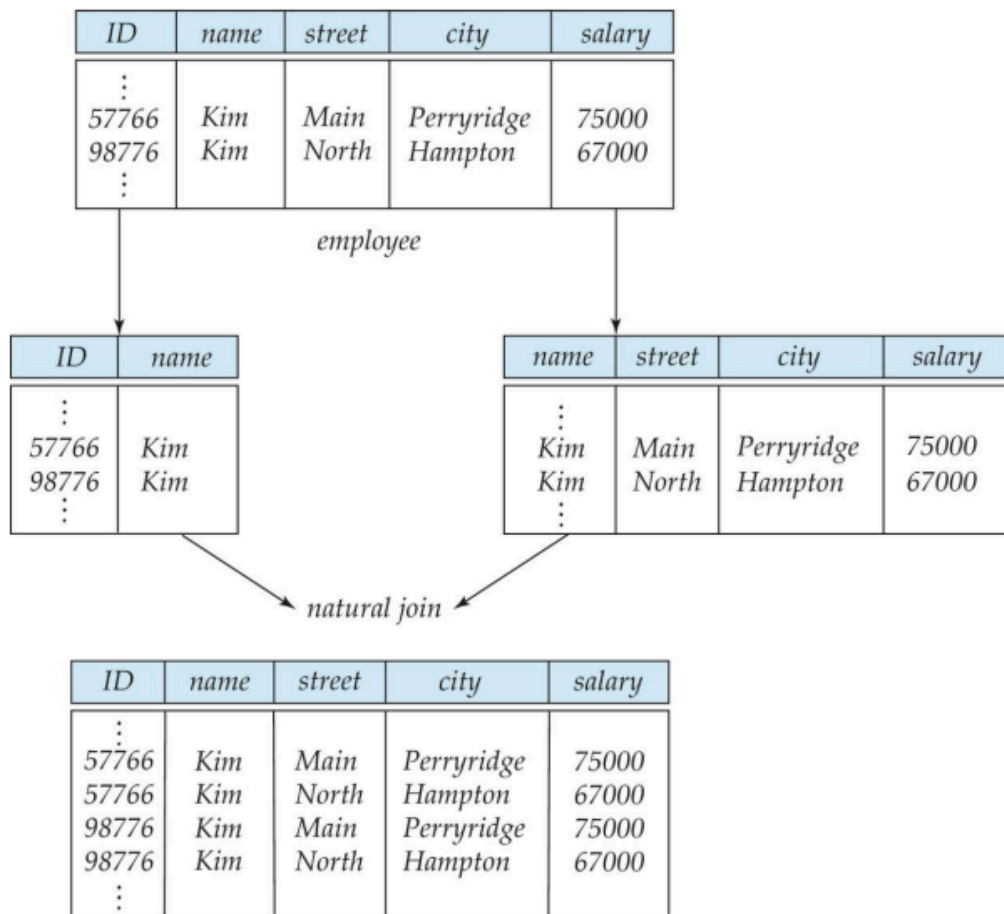
The issue arises because **name** is used as the common attribute between the two decomposed tables, but names are not guaranteed to be unique. Therefore, when trying to join `employee1` and `employee2` back together based on `name`, if two employees share the same name, it becomes impossible to correctly match up rows from the two tables, leading to **loss of information**.

Key Points:

- **Lossy Decomposition:** This is an example of a **lossy decomposition**, meaning that important information from the original relation is lost in the process of decomposition, and the original relation cannot be accurately reconstructed from the decomposed relations.

- **Uniqueness of Identifiers:** The figure emphasizes that using non-unique attributes (like names) for decomposition can lead to incorrect results, as multiple rows may refer to the same attribute value (like two employees with the same name).

A Lossy Decomposition



Page 7.7: Lossless Decomposition

- **Definition:** A decomposition of a relation R into R_1 and R_2 is **lossless** if, after performing the natural join between R_1 and R_2 , the result is exactly the same as the original relation R .

- **Formal Representation:**

For a relation R , decomposed into R_1 and R_2 , the decomposition is lossless if:

$$\pi_{R_1}(r) \bowtie \pi_{R_2}(r) = r$$

where r is the instance of the relation R , π_{R_1} and π_{R_2} are the projections of r onto R_1 and R_2 , respectively, and $\bowtie$ is the natural join operation.

- **Conditions for Losslessness:** For a decomposition of R into R_1 and R_2 , the decomposition is lossless if at least one of the following holds:
 - $R_1 \cap R_2 \rightarrow R_1$
 - $R_1 \cap R_2 \rightarrow R_2$

This means that the attributes common to both R_1 and R_2 must functionally determine the rest of the attributes in at least one of the decomposed relations.

The example on this page shows a simple case to demonstrate a lossless decomposition.

- **Relation $R(A, B, C)$** is decomposed into two smaller relations:
 - $R_1(A, B)$
 - $R_2(B, C)$

Here, the attribute B is the common attribute between R_1 and R_2 . The decomposition is **lossless** because:

- $B \rightarrow A$ in R_1
- $B \rightarrow C$ in R_2

Therefore, when performing a natural join on R_1 and R_2 , we can reconstruct the original relation R without losing any information.

Key Points:

1. **Lossless vs. Lossy:** Lossless decomposition ensures that the original relation can be perfectly reconstructed after decomposition, while lossy decomposition leads to loss of information.
2. **Functional Dependency:** The key to ensuring a lossless decomposition is that the common attribute(s) between the decomposed relations must functionally determine the rest of the attributes in at least one of the relations.

$$\pi_{R_1}(r) \bowtie \pi_{R_2}(r) = r$$

The expression shown is a formal way to represent a **lossless join decomposition** in relational databases. Here's how you can read and understand it:

Explanation of Notation:

- $\pi_{R_1}(r)$: This denotes the **projection** of the relation r onto the attributes of the sub-relation R_1 . A projection essentially selects certain columns (attributes) from a relation. In this case, it selects the attributes that are present in R_1 from the original relation r .
- $\pi_{R_2}(r)$: Similarly, this represents the **projection** of the relation r onto the attributes of the sub-relation R_2 . It selects the columns present in R_2 .
- $\bowtie$ (**natural join**): This symbol represents the **natural join** operation, which combines the two projected sub-relations R_1 and R_2 based on common attributes. In a natural join, the rows from R_1 and R_2 are joined when they have the same values for the common attributes.
- r (on the right-hand side): This is the original relation that was decomposed into R_1 and R_2 . The equation says that after projecting r into R_1 and R_2 , and then joining them, you should get back the original relation r .

Reading the Equation:

- "The projection of r onto R_1 , joined with the projection of r onto R_2 , results in the original relation r ."

Functional Dependency:

It shows how one attribute uniquely determines another attribute in a relation.

Let R be a relation, and let X and Y be subsets of attributes of R . A functional dependency $X \rightarrow Y$ holds if, for any two tuples (rows) in the relation R , if those tuples have the same values for the attributes in X , then they must also have the same values for the attributes in Y .

- $X \rightarrow Y$ means:
 - If two rows agree on the values of X , they must agree on the values of Y

Example:

Consider a relation `Employee(EmployeeID, Name, Department, Salary)` with the following tuples:

EmployeeID	Name	Department	Salary
101	Alice	HR	60000
102	Bob	IT	70000
101	Alice	HR	60000

In this case, we have the following **functional dependency**:

- $\text{EmployeeID} \rightarrow \text{Name, Department, Salary}$

This means that if two tuples have the same `EmployeeID`, they must also have the same `Name`, `Department`, and `Salary`. The `EmployeeID` uniquely determines the other attributes in the relation.

Functional Dependency Notation:

- $X \rightarrow Y$: Attribute X functionally determines attribute Y.
 - Determinant: X is the left-hand side (the set of attributes that determine the other set).
 - Dependent: Y is the right-hand side (the set of attributes being determined).

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α , they also agree on the attributes β . That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

A **trivial functional dependency** is a type of functional dependency in a relational database where a set of attributes functionally determines itself or a subset of its attributes. In other words, if we have an attribute set X , a functional dependency of the form $X \rightarrow X$ or $X \rightarrow Y$, where Y is a subset of X , is considered trivial.

Examples:

1. **Example 1:** For a relation with attributes $\{A, B\}$, the functional dependency $\{A, B\} \rightarrow A$ is trivial because A is a subset of $\{A, B\}$.
2. **Example 2:** The dependency $\{A\} \rightarrow A$ is also trivial, since A functionally determines itself.

Closure of a Set of Functional Dependencies:

Imagine you have a set of functional dependencies (FDs). A closure of this set is all the additional functional dependencies you can logically infer from the original ones.

- The closure is represented as F^+ if the original set of dependencies is F .

Why do we need a closure?

When designing or normalizing databases, we often decompose relations into smaller tables. To ensure no important data relationships are lost during decomposition, we need to know all the functional dependencies that apply to the data. The closure gives us all the functional dependencies, including those implied by the original ones.

2. Basic Idea of Inference

To explain this, let's think about a simple example.

Example:

We have a relation $R(A, B, C)$ with a set of functional dependencies:

- $A \rightarrow B$
- $B \rightarrow C$

Here's how closure works:

If we know that $A \rightarrow B$ and $B \rightarrow C$, we can infer that $A \rightarrow C$ by using transitivity (just like in math: if A leads to B, and B leads to C, then A must lead to C).

So, the closure of the set $F = \{A \rightarrow B, B \rightarrow C\}$ is:

- $A \rightarrow B$ (original)
- $B \rightarrow C$ (original)
- $A \rightarrow C$ (inferred)

This set of all possible functional dependencies is called the closure F^+ .

Superkey

A **superkey** is a set of one or more attributes (columns) in a relation (table) that can **uniquely identify a tuple (row)** in the relation. In simpler terms, if you know the values of all the attributes in a superkey, you can always identify a unique row in the table.

Example of a Superkey:

Let's say we have a table `in_dep` with the following attributes:

ID	Name	Salary	Dept_Name	Building	Budget
101	Alice	70,000	CS	Bldg 1	200,000
102	Bob	60,000	Math	Bldg 2	150,000
103	Eve	80,000	CS	Bldg 1	200,000

In this table:

- The attribute **ID** is a superkey because it **uniquely identifies each row**. No two rows have the same **ID**, so knowing the **ID** of a row will always allow you to find that exact row.
- A combination of attributes, like **ID**, **Name**, is also a superkey because **ID** alone already uniquely identifies a row. In other words, adding more attributes to a superkey does not change the fact that it uniquely identifies a row.

Formally, for a relation schema **R**, a set of attributes **K** is a **superkey** if $K \rightarrow R$.

This means that knowing the values of the attributes in **K**, you can determine the values of all other attributes in the relation schema.

Candidate Key

A **candidate key** is a **minimal superkey**. It is a superkey that **cannot have any attributes removed** without losing its ability to uniquely identify rows. In other words, a candidate key is the simplest combination of attributes that uniquely identifies each row in a table.

Example of a Candidate Key:

In the **in_dep** table:

- The attribute **ID** is a **candidate key** because it uniquely identifies each row and there's no smaller subset of attributes that can do the same.
- The combination **ID**, **Name** is **not** a candidate key because **ID** alone is sufficient to identify a row. So, **ID**, **Name** is a superkey but not a candidate key since it has unnecessary attributes (**Name**).

Formally, a set of attributes **K** is a **candidate key** if:

1. $K \rightarrow R$ (it is a superkey).
 2. There is no subset of **K** that is also a superkey (this makes it minimal).
-

Functional Dependencies and Superkeys

Functional dependencies play an important role in identifying keys. A **functional dependency** $\alpha \rightarrow \beta$ means that the values of the attributes in α uniquely determine the values of the attributes in β .

For a set of attributes K to be a **superkey**, it must satisfy the functional dependency $K \rightarrow R$, which means the values of K must determine the values of all the other attributes in the relation schema R . If a set of attributes K satisfies this condition and there is no smaller subset of K that also satisfies it, then K is a **candidate key**.

Example: Keys and Functional Dependencies in a Database Schema

Let's consider a more specific example. Suppose we have a relation schema `in_dep` with the attributes:

- `ID` (Instructor ID)
- `Name` (Instructor Name)
- `Salary`
- `Dept_Name` (Department Name)
- `Building` (Department's Building)
- `Budget` (Department's Budget)

We can expect the following functional dependencies in the schema:

- $\text{Dept_Name} \rightarrow \text{Building}$: The department name determines the building (each department is associated with one building).
- $\text{ID} \rightarrow \text{Name, Salary, Dept_Name, Building, Budget}$: The instructor's ID determines the instructor's name, salary, department, building, and budget (since each instructor has a unique ID, it can be used to find all the other information).

Identifying Superkeys and Candidate Keys:

- **Superkey:** The attribute **ID** is a superkey because it uniquely identifies each instructor and all the associated information (Name, Salary, Dept_Name, Building, Budget).
- **Candidate Key:** Since **ID** is minimal (we can't remove any attribute from it without losing its ability to uniquely identify a row), **ID** is also a **candidate key**.

Additionally, if we consider **Dept_Name**:

- The dependency **Dept_Name** $\rightarrow$ **Building** tells us that each department is in one specific building, but this doesn't uniquely identify instructors or other information about them. Therefore, **Dept_Name** alone is **not a candidate key**.

. Using Functional Dependencies to Express More Constraints

Functional dependencies allow us to express certain constraints that go beyond simple keys. For example:

- **Dept_Name** $\rightarrow$ **Building** tells us that the department name determines the building, but this does not make **Dept_Name** a superkey, because multiple instructors can belong to the same department, and **Dept_Name** alone does not uniquely identify them.
- **ID** $\rightarrow$ **Building** means that knowing the instructor's ID can also tell us the building they work in (because ID uniquely determines the department, and the department determines the building).

However, this is not a typical dependency you would use, because **ID** $\rightarrow$ **Dept_Name** $\rightarrow$ **Building** is already covered by transitivity.

Normalization

Normalization in database design is a process used to organize a database into tables and columns to reduce redundancy and improve data integrity. The main goals of normalization are to:

1. **Eliminate redundant data:** Ensuring that each piece of data is stored only once.
2. **Ensure data dependencies make sense:** Data is logically stored, which helps in maintaining data integrity.

Normalization typically involves dividing a database into two or more tables and defining relationships between the tables. [This process is guided by a series of rules known as “normal forms,” each addressing specific types of redundancy and dependency issues](#)¹².

Here are the key normal forms:

1. **First Normal Form (1NF):** Ensures that the table has a primary key and that all columns contain atomic (indivisible) values.
2. **Second Normal Form (2NF):** Builds on 1NF by ensuring that all non-key attributes are fully functionally dependent on the primary key.
3. **Third Normal Form (3NF):** Ensures that all the attributes are functionally dependent only on the primary key and not on any other non-key attributes.

Dependency Preservation:

Dependency preservation is a key concept in relational database design, especially when dealing with **normalization** and **decomposition**. It ensures that when a database relation (table) is broken down into smaller relations (for better design), the important data relationships (i.e., **functional dependencies**) are not lost or made more difficult to enforce.

Let's break this down in detail:

1. What is Dependency Preservation?

Dependency preservation means that after decomposing a table into smaller tables, all the functional dependencies that existed in the original table can still be **enforced** by looking at the individual smaller tables, without needing to join them back together.

In simple terms:

- When we decompose a table into two or more smaller tables, we want to ensure that **all the rules (functional dependencies)** governing how the data is related are still **checkable** in the smaller tables.

- If you can still enforce all the functional dependencies by looking at each decomposed table separately, the decomposition is said to be **dependency preserving**.

2. Why is Dependency Preservation Important?

- When you update a database (e.g., insert, delete, or modify records), the database management system (DBMS) needs to **enforce the functional dependencies** to maintain data integrity.
- If a decomposition **does not preserve dependencies**, you might need to **join** the decomposed tables back together to check some of the functional dependencies. This process can be computationally expensive and slow down the system.
- A decomposition that **preserves dependencies** allows for easy checking of functional dependencies without needing to join tables, ensuring **efficient enforcement** of data rules.

3. Example of Dependency Preservation

Let's use an example to make this clearer.

Original Table:

Suppose we have a relation **Employee** with the following attributes:

- **Employee(ID, Name, Dept_Name, Salary)**

And we have these functional dependencies:

- **ID → Name, Dept_Name, Salary** (Each employee's ID uniquely determines their name, department, and salary)
- **Dept_Name → Salary** (Each department has a fixed salary range, so the department determines the salary)

Decomposition:

Now, if we decompose this relation into two smaller relations:

1. **Employee1(ID, Name, Dept_Name)** – stores employee ID, name, and department.

2. `Employee2(Dept_Name, Salary)` – stores the department and its salary.

Dependency Preservation:

- The first functional dependency `ID → Name, Dept_Name, Salary` is preserved because in the `Employee1` table, `ID` still determines `Name` and `Dept_Name`. And in the `Employee2` table, `Dept_Name` determines `Salary`.
- The second functional dependency `Dept_Name → Salary` is preserved in the `Employee2` table.

Therefore, in this case, we can **enforce all functional dependencies** by looking at the individual decomposed tables, meaning the decomposition is **dependency preserving**.

4. When is a Decomposition Not Dependency Preserving?

A decomposition is **not dependency preserving** if we **cannot enforce** all functional dependencies by just looking at the smaller tables separately. In such cases, some dependencies might only be checkable if we join the decomposed tables back together.

Example of Non-Dependency Preservation:

Let's modify the previous example. Suppose we decompose `Employee(ID, Name, Dept_Name, Salary)` into:

1. `Employee1(ID, Name)` – stores employee ID and name.
2. `Employee2(Dept_Name, Salary)` – stores department and salary.

Now:

- We still have `ID → Name` in `Employee1`, and `Dept_Name → Salary` in `Employee2`.
- But the functional dependency `ID → Dept_Name, Salary` (which states that an employee's ID determines their department and salary) **cannot be enforced** just by looking at the individual tables.

To check $ID \rightarrow Dept_Name, Salary$, you would need to join **Employee1** and **Employee2** back together. This means the decomposition is **not dependency preserving** because we lost the ability to enforce one of the original functional dependencies without doing a join operation.

5. Dependency Preservation in Normalization

During **normalization** (the process of decomposing tables to reduce redundancy and avoid anomalies), it is important to achieve two goals:

1. **Lossless join:** The original relation should be reconstructable from the decomposed relations without losing any information.
2. **Dependency preservation:** All the functional dependencies in the original relation should still be enforceable without needing to join the decomposed tables.

6. How to Check for Dependency Preservation?

To check if a decomposition preserves dependencies:

- For each functional dependency in the original set, check if it can be enforced by looking only at the decomposed relations.
- If you can enforce all dependencies without joining the tables, the decomposition is dependency preserving.